NASA SPACE APPS CHALLENGE

Ad Astra Exosky

Team Members
Rita Hochar
David Atanalian
Shady Aksamawati
Andy Kayrouz
Charbel Jabbour
Leo-George Chlala

2024

Contents

1 Introduction **2**
1.1 Overview 2
1.2 Objectives 2

2 Methodology **3**
2.1 Data Acquisition 3
2.1.1 Star Data Import 3
2.1.2 Coordinate Transformation 4
2.1.3 Magnitude Calculations 5
2.1.4 Data Export 5
2.2 Application Development 6
2.2.1 Unity Integration 6
2.2.2 User Interface 11
2.2.3 Educational Content 11

3 Conclusion **12**

4 Resources **12**

1 Introduction

1.1 Overview

This project aimed to develop an application that allows users to visualize the night sky as seen from various exoplanets within a given solar system. By utilizing data from the NASA Exoplanet Archive and modern star catalogs, the application provides an engaging way for users to explore the stars visible from selected exoplanets, enhancing their understanding of astronomy through interactive features.

1.2 Objectives

- Create an interface for users to choose an exoplanet and visualize the corresponding night sky.
- Display an interactive star chart based on the star data relevant to the selected solar system.
- Enable users to export high-quality images for printing or virtual reality viewing.
- Make astronomy engaging and accessible for audiences of all backgrounds and ages.

2 Methodology

2.1 Data Acquisition

We used Python to calculate the needed data for the night view. The main purpose of this code is to transform the coordinates of celestial objects from a galactic coordinate system to a new Cartesian coordinate system centered at a specified point (the exoplanet).

2.1.1 Star Data Import

The Gaia open-source library in Python was used to import essential star data, including coordinates, brightness, and parallax.

```
from astroquery.gaia import Gaia
query = """
SELECT TOP 1000000
    source_id,  -- Unique Gaia identifier (star name)
    ra,         -- Right Ascension
    dec,        -- Declination
    phot_g_mean_mag,  -- G-band magnitude (intensity)
    parallax    -- Parallax value (mas)
FROM gaiadr3.gaia_source
WHERE phot_g_mean_mag < 15
"""
job = Gaia.launch_job(query)
results = job.get_results()
```

Code Listing 1: Data from Gaia

2.1.2 Coordinate Transformation

Converted star coordinates from a Galactic system to a Cartesian coordinate system.

```
def galactic_to_cartesian(b, l, r):
    """Convert galactic coordinates (b, l, r) to Cartesian
   coordinates (x, y, z)"""
    b_rad = np.radians(b)
    l_rad = np.radians(l)

    x = r * np.cos(b_rad) * np.cos(l_rad)
    y = r * np.cos(b_rad) * np.sin(l_rad)
    z = r * np.sin(b_rad)

    return np.array([x, y, z])
```

Code Listing 2: Galactic to Cartesian

Applied translations and rotations to reposition stars relative to the chosen exoplanet, treating it as the new origin.

```
 #Convert to Cartesian coordinates
 v = galactic_to_cartesian(b, l, r)
 #Calculate the rotation angles
 theta_z = np.radians(10)  # Rotation around z-axis
 theta_y = np.radians(90 - b0)  # Rotation around y-axis
# Define the new center coordinates in Cartesian
new_center = galactic_to_cartesian(new_center_b, new_center_l
   , new_center_distance)
 # Transform the galactic coordinates to the new system
 transformed_cartesian = transform_galactic_coordinates(b, l,
    distance, new_center_l, new_center_b)
 # Apply the translation to move the origin to the new center
 final_coordinates = transformed_cartesian + new_center
```

Code Listing 3: Transformation

2.1.3 Magnitude Calculations

Calculated the apparent magnitude of stars using absolute magnitude and distance from the new reference point.

```
def calculate_distance_from_cartesian(x, y, z):
    """
    Calculate the Euclidean distance from the origin (0, 0, 0) using the
    Cartesian coordinates (x, y, z).
    """
    return np.sqrt(x**2 + y**2 + z**2)

   def calculate_apparent_magnitude(M, distance):
    """
    Calculate the apparent magnitude using the distance modulus formula.

    M: Absolute magnitude (g-band magnitude given)
    distance: Distance to the star in parsecs
    """
    return M + 5 * np.log10(distance) - 5
```

Code Listing 4: Apparent Magnitude

Filtered out stars with a magnitude greater than 6 to ensure only those visible to the naked eye were represented, enhancing the realism of the night sky.

2.1.4 Data Export

Compiled the filtered star data into a CSV file for integration with Unity.

2.2 Application Development

2.2.1 Unity Integration

1. **Star Plotting**
   Imported the CSV data using a C♯ code into Unity to create a 3D visualization of the stars, allowing users to explore the night sky.

```
using UnityEngine;
using System.IO;
public class CSVDataPlotter : MonoBehaviour
{
    public GameObject plotPrefab;
    public string filePath = "filtered.csv";
    public float scaleFactor = 5.0f;

    void Start() => LoadAndPlotCSV();
    void LoadAndPlotCSV()
    {
        if (!File.Exists(filePath)) return;
        foreach (var row in File.ReadAllLines(filePath)
    [1..])
        {
            var cols = row.Split(',');
            if (cols.Length < 6) continue;
            var position = new Vector3(
                float.Parse(cols[0]) * scaleFactor,
                float.Parse(cols[1]) * scaleFactor,
                float.Parse(cols[2]) * scaleFactor
            );
            Instantiate(plotPrefab, position, Quaternion.
    identity);
        }
    }
}
```

Code Listing 5: Plotting

2. **Camera Controller**

   Wrote a code to implement movement controls across the night sky. Using WASD the user can move in all directions.

```
using UnityEngine;
public class CameraAndGroundController : MonoBehaviour
{
    public float rotationSpeed = 150f;
    public GameObject ground;
    private float pitch = 0f, yaw = 0f;

    void Update()
    {
        pitch += (Input.GetKey(KeyCode.S) ? 1 : Input.GetKey(KeyCode.W) ? -1 : 0) * rotationSpeed * Time.deltaTime;
        yaw += (Input.GetKey(KeyCode.A) ? -1 : Input.GetKey(KeyCode.D) ? 1 : 0) * rotationSpeed * Time.deltaTime;
        transform.eulerAngles = new Vector3(Mathf.Clamp(pitch, -90f, 90f), yaw, 0f);
        ground?.SetActive(pitch < 30f);
    }
}
```

Code Listing 6: Camera And Ground Controller

3. **Planet Orbits**

   Simulating the orbiting paths and orbit lines of the planets around their stars. User can turn on/off the orbit by clicking the "space" button.

```
using UnityEngine; using System.Collections;

public class E : MonoBehaviour
{
    public Transform c; public float a = 10, b = 5, s =
   1; private float t;
    void Update() { if (Input.GetKeyDown(KeyCode.Space))
   StartCoroutine(O()); }
    IEnumerator O()
    {
        while (true) { t += Time.deltaTime * s; transform
   .position = new Vector3(a * Mathf.Cos(t), 0, b * Mathf
   .Sin(t)) + c.position; yield return null; }
    }
}
```

Code Listing 7: Elliptical Orbit

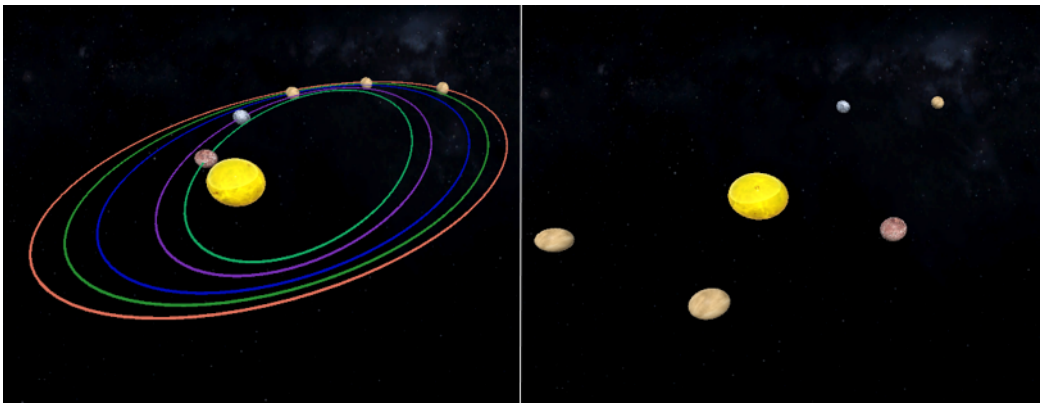


Figure 1: Orbits

```
using UnityEngine;
[RequireComponent(typeof(LineRenderer))]
public class OrbitLines : MonoBehaviour
{
    public Transform centerObject; public float a = 5f, b = 3f; public int s = 100; private LineRenderer lr;

    void Start()
    {
        lr = GetComponent<LineRenderer>(); lr.startWidth = lr.endWidth = 0.1f; lr.startColor = lr.endColor = Color.white; lr.material = new Material(Shader.Find("Sprites/Default"));
    }
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space)) lr.enabled = !lr.enabled;
        if (lr.enabled) DrawOrbit();
    }

    void DrawOrbit()
    {
        var p = new Vector3[s + 1];
        for (int i = 0; i <= s; i++)
            p[i] = new Vector3(Mathf.Cos(i * 2 * Mathf.PI / s) * a, 0, Mathf.Sin(i * 2 * Mathf.PI / s) * b) + centerObject.position;
        lr.SetPositions(p);
    }
}
```

Code Listing 8: Orbit Lines

4. **Telescope Interaction**

This code manages the interaction between a telescope and the user camera in Unity, allowing the user to toggle between views and adjust the telescope's zoom using mouse scroll input.

```
using UnityEngine;
public class T : MonoBehaviour
{
    public GameObject tCam, pCam, tObj;
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.E))
    {
            bool v = tCam.activeSelf;
            tCam.SetActive(!v); pCam.SetActive(v);
            foreach (var x in tObj x.enabled = v;
        }
        if (tCam.activeSelf) tCam.GetComponent<Camera>().
   fieldOfView = Mathf.Clamp(tCam.GetComponent<Camera>().
   fieldOfView - Input.GetAxis("Mouse ScrollWheel") * 10,
    20, 60);
}
```

Code Listing 9: Telescope Interaction

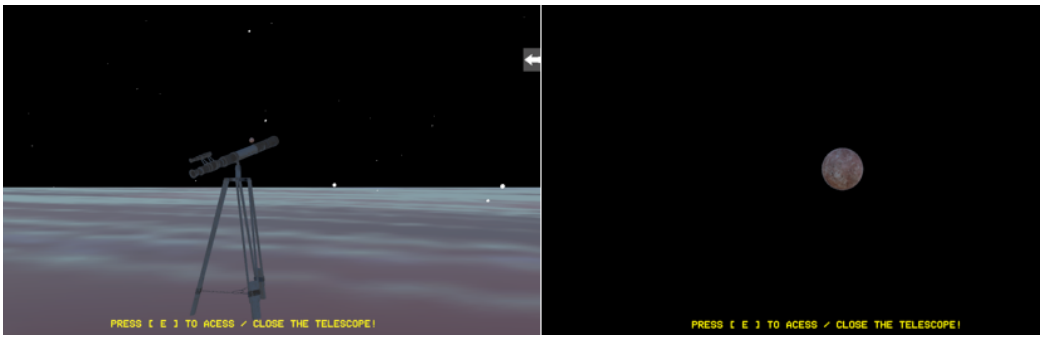


Figure 2: Telescope

2.2.2 User Interface

Developed a user-friendly interface for selecting an exoplanet and displaying the corresponding star chart. The user will have a collection of 10 stars to pick from. When a user selects a star, they will be directed to a general page with information about the star and a button that allows them to access the night sky view. ItâĂŹs important to note that the night sky view will be identical from all planets within the same solar system. Additionally, users can zoom in using a telescope for a closer look at celestial objects. They also have the option to create and name their own constellations, enhancing their experience of star mapping. When viewing through the telescope, users will have the opportunity to observe the planets in the star system, making for an engaging and interactive exploration of the cosmos.

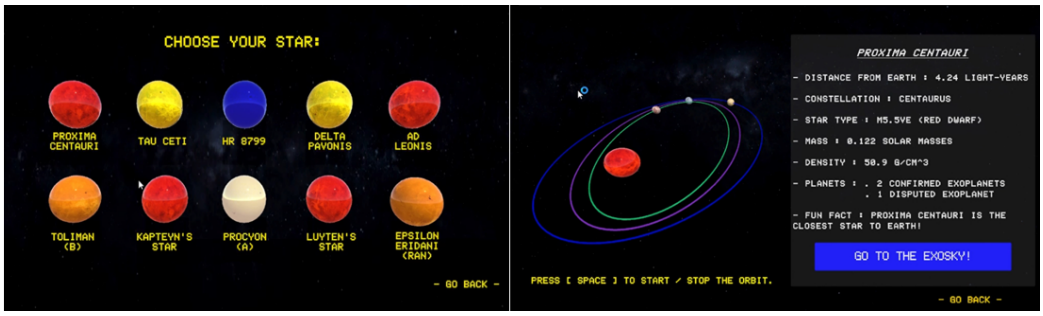


Figure 3: Menu - Info Page

2.2.3 Educational Content

Provided contextual information about the solar system from which the stars are viewed, reinforcing the educational aspect of the application. Additionally, This allows astronomers to analyze star positions relative to a different reference point, facilitating various astronomical studies, such as the dynamics of star clusters or the galaxy's structure.

3 Conclusion

This project highlights the effective integration of astronomy and technology to create engaging educational tools. By enabling users to experience the night sky from various exoplanets, the application fosters curiosity and promotes learning in an accessible format. Future enhancements could include additional interactive elements or user-generated content to further enrich the educational experience.

4 Resources

Gaia: Open-source library for astronomical data.
NASA Exoplanet Archive: Database of discovered exoplanets and related data.
Fundamental Astronomy: Karttunen et al., 2017, 6th ed., Springer. This book provided foundational knowledge on astronomical concepts and methodologies used throughout the project.